

RELATIONAL SEPARATION LOGIC FOR COMPILER VERIFICATION

MPRI Internship Report

Gabriel DESFRENE

Supervised by Xavier LEROY and François POTTIER

General Context

Modern software relies on high-level languages and formal verification techniques, such as deductive verification, model checking, or static analysis, to establish correctness guarantees. Hardware executes machine code produced by compilers, which translate source programs through intermediate representations. Consequently, source-level correctness guarantees remain meaningful only if compilation strictly preserves program semantics. Unverified compilation passes risk introducing bugs or invalidating source-level formal proofs.

Initial efforts in verified compilation established foundational methodologies for proving semantic preservation. The CompCert compiler [Ler09b] relies on simulation diagrams, establishing a correspondence between execution steps of the source and target programs while maintaining a state invariant. CakeML [Tan+19], a verified compiler for functional languages, combines simulation diagrams with equational reasoning to prove its correctness. Beyond general-purpose compilers, other verified compiler projects focus on specialized application domains. For instance, Jinja [KN06] targets object-oriented programs compiled to the Java Virtual Machine, whereas Jasmin [Alm+17] verifies a dedicated compiler for high-performance cryptographic software.

Existing verification paradigms exhibit limitations. On the one hand, equational reasoning requires programs to be sufficiently similar. When two programs are written in different languages, use distinct memory representations, or follow different calling conventions, equality may no longer be applicable to state the soundness of the transformation. One must instead establish that target behaviors safely refine source behaviors. On the other hand, traditional simulation diagrams grow complex because they require constructing a global invariant upfront. This invariant must relate entire machine configurations and whole memory states across all program points.

Recent developments in formal methods provide techniques to address these issues. Modern coinductive simulation frameworks, such as the *Stuttering for Free* framework by Cho et al. [Cho+23] or *Interaction Trees* [Xia+20], simplify reasoning about pairs of programs whose execution steps do not correspond one-to-one. In parallel, *relational separation logics* enable local reasoning across dual executions. They allow memory resources to be tracked, partitioned, and treated independently between source and target heaps, avoiding reasoning on global memory injections. For instance, Simuliris [Gäh+22] uses a relational separation logic to verify source-to-source program transformations, and Allain et al. [All+25] employ relational separation logic to validate the Tail Modulo Cons optimization.

Research Problem

In verified compilation, semantic preservation is ultimately characterized by a backward simulation. This ensures that every observable behavior of the target program is permitted by the source program. However, existing compilers such as CompCert and CakeML primarily establish forward simulations. Forward simulations are simpler to construct, but they only imply backward simulations when the target language is deterministic. Supporting target non-determinism, such as non-deterministic assignments or concurrency, requires establishing a backward simulation directly.

Furthermore, existing verified compilers rely on global memory invariants and whole-program simulation relations. Using a relational separation logic offers a promising alternative: it allows transformations to be verified by reasoning locally about modified heap cells without considering the entire memory.

This internship investigates whether recent developments in simulation relations and separation logic can be integrated into a relational separation logic framework and used to verify compilers. To our knowledge, no existing verified compiler incorporates a relational separation logic to prove the soundness of its optimization passes. In short, this approach relaxes constraints present in existing systems, such as the determinism requirements imposed by CompCert and CakeML.

Contributions

We developed a mechanized logic framework addressing these challenges. At its core, we use a coinductive safe backward simulation relation. This frees our approach from determinism restrictions on the source and target languages. The simulation relation is asymmetric: it permits target executions to exploit undefined behaviors in the source, enabling optimizations that assume source execution does not get stuck.

Building upon this simulation relation, we defined a relational separation logic that enables local reasoning over fragments of source and target heaps. The logic is parameterized by the operational semantics of the source and target languages under comparison. Our framework uses this logic to specify and verify relational judgments between program components.

We instantiated this framework with a low-level imperative language, μ RTL, based on the Register Transfer Language used in CompCert. We formalized the operational semantics for this μ RTL dialect. It features constructs for arithmetic operations, dynamic heap allocation and deallocation, and function calls.

Finally, we established a system of relational Hoare rules for μ RTL programs. This proof system comprises two distinct categories of rules. Asynchronous rules govern unilateral execution steps taken by either the source or the target program. These rules enable target-side optimizations by exploiting the absence of source undefined behavior. Synchronous rules govern joint execution steps when both programs advance simultaneously on matching instructions, such as two calls to the same function.

Arguments Supporting Its Validity

We evaluated our framework by verifying the correctness of two concrete program transformations and two general compiler optimizations. For the concrete transformations, the first exploits the absence of undefined behavior in the source to eliminate a redundant zero-check after a division, while the other hoists an invariant memory read out of a loop body. For the general optimizations, the first is a branch tunneling pass that eliminates redundant jumps across arbitrary control-flow graphs. The second, which is omitted from the detailed exposition of this report, optimizes redundant instructions, such as superfluous register copies.

Beyond these case studies, we established metatheoretic properties confirming framework soundness. We proved that our relational separation logic soundly implies the underlying backward simulation relation, ensuring semantic preservation. Additionally, we proved that our simulation formulation, which is easier to work with, is equivalent to standard backward simulation definitions used in the literature [Ler09b; Gäh+22; All+25].

The entire framework has been mechanized in the Rocq proof assistant [The26]. The development relies on the coinduction Rocq library [Pou26] and the `stdpp` library [Jun+26]. Furthermore, our logic is compatible with the Iris proof mode [Kre+18].

Future Work

This work provides a first step for several research perspectives. While current case studies focus on control-flow simplifications and local rewrites, future work will target data-flow optimizations, ranging from constant propagation to more complex transformations such as selective defunctionalization.

A second direction involves multi-language verification and language independence. Although our logic is parameterized to relate distinct languages, our current case studies operate within a single intermediate representation. Future work will apply the framework to translation passes between different intermediate languages. More generally, we aim to keep the relational separation logic as language-independent as possible.

Another objective is full-scale integration into CompCert. A key question is whether our relational verification technique can be applied incrementally, starting with a single compilation pass and progressively extending to other passes across the pipeline.

Finally, we aim to support richer language features. While our logic framework accommodates non-determinism, our current μ RTL semantics features only limited non-determinism from memory allocation. We plan to investigate languages featuring concurrency, structured control, first-class closures, or effect handlers. In the long term, these techniques could be applied to extend CompCert with new optimizations and shared-memory concurrency, paving the way toward a verified OCaml compiler.

Thanks

Many thanks to my supervisors Xavier LEROY and François POTTIER for their guidance throughout this internship.

1 Verified Compilation Background

1.1 Small-Step Operational Semantics

Giving a formal semantics to a programming language provides a rigorous meaning to its programs. Small-step operational semantics models computation as a sequence of elementary transitions between program configurations. Rather than evaluating compound statements in a single step, it describes how internal configurations evolve under primitive operations. This style directly captures non-termination as infinite transition sequences and permits the inspection of intermediate configurations.

Formally, a small-step operational semantics is defined by a transition system over a set of configurations C , equipped with a binary reduction relation $\rightarrow \subseteq C \times C$. In our framework, a configuration c is represented as a pair $c = (s, m) \in S \times M$, where $s \in S$ denotes an internal program state and $m \in M$ denotes a memory state. The structure of the internal state s depends on the language under study: for instance, s may be a term in a λ -calculus with substitution semantics, or, as in the μ RTL language introduced later in this report, a program, a call stack, a currently active function, a program counter, and an environment.

The transition relation $(s, m) \rightarrow (s', m')$ indicates that the configuration (s, m) reduces in a single step to (s', m') . The global program code and declarations are implicitly fixed within the transition relation. We denote by $\rightarrow^*$ the reflexive-transitive closure of $\rightarrow$. The transition system need not be deterministic: a configuration may admit several outgoing transitions.

To characterize execution endpoints, the semantics is parameterized by a set of observable values val and a partial function is_final . A configuration c is *final* when $\text{is_final}(c) = \text{Some}(v, m)$ for some return value $v \in \text{val}$ and memory $m \in \text{memory}$. For such final configurations, no further reduction steps are possible: if $\text{is_final}(c) = \text{Some}(v, m)$, there exists no configuration c' such that $c \rightarrow c'$.

We say that a configuration c can make progress, written $\text{can_progress}(c)$, if there exists a configuration c' such that $c \rightarrow c'$. Conversely, we follow the convention that a configuration c is *stuck*, written $\text{stuck}(c)$, if it is not a final configuration and cannot perform any transition. Stuck configurations represent execution failures where the operational semantics is undefined. In an imperative setting, a configuration becomes stuck when dereferencing an unallocated pointer, dividing by zero, or branching to an invalid control-flow node.

1.2 Program Behaviors

Equipped with a small-step operational semantics, we can formally classify the observable behaviors of a program execution. Because the transition system may be non-deterministic, an execution path starting from a configuration c can yield multiple outcomes. The syntax of observable behaviors b is defined by the following grammar:

$$b \in \text{behavior} ::= \text{Terminating}(v, m) \mid \text{Diverging} \mid \text{Undefined}$$

where $v \in \text{val}$ and $m \in \text{memory}$. We write $b \in c$ to say that the configuration c exhibits the behavior b . We say that:

- c *terminates* with value v and memory m if there is a final configuration c' such that $c \rightarrow^* c'$ and $\text{is_final}(c') = \text{Some}(v, m)$.
- c *diverges* if there exists an infinite reduction sequence starting from it. We do not restrict divergence to strongly diverging configurations, where every execution path is infinite.
- c exhibits *undefined behavior* if there is a configuration c' such that $c \rightarrow^* c'$ and $\text{stuck}(c')$.

The notion of behavior is extended to a complete program ($b \in P$) by considering the behaviors of its initial state. Every configuration admits at least one observable outcome:

Lemma 1. For any configuration c , there exists a behavior b such that $b \in c$.

1.3 Program Refinement

The objective of a verified compiler is to guarantee semantic preservation: the compiled target executable must faithfully preserve the behaviors of the source program. In verified compilation, semantic preservation is formalized as a *behavioral refinement* between the compiled target program T and the source program S .

A direct formulation is the *rigid behavioral refinement*, requiring every behavior of the target program to be an admitted behavior of the source program:

$$T \sqsubseteq S \triangleq \forall b, b \in T \implies b \in S \quad (1)$$

Rigid refinement ensures that compilation never introduces spurious behaviors, while permitting the compiler to resolve non-deterministic choices present in the source. However, the rigid behavioral refinement of eq. (1) is overly restrictive for optimizing compilers. If a source program only exhibits undefined behaviors, an optimizing compiler may eliminate an erroneous code path or transform it into a benign execution for the target.

To account for these optimization principles, compiler correctness relies on a *flexible behavioral refinement*. We first equip behaviors with a preorder $\leq_{\text{beh}}$, parameterized by a relation on values and memory Φ :

$$\begin{array}{c} \frac{\Phi(v_t, v_s, m_t, m_s)}{\text{Terminating}(v_t, m_t) \leq_{\text{beh}} \text{Terminating}(v_s, m_s)} \text{ BEH-TERM} \qquad \frac{}{\text{Diverging} \leq_{\text{beh}} \text{Diverging}} \text{ BEH-DIV} \\[10pt] \frac{}{b_t \leq_{\text{beh}} \text{Undefined}} \text{ BEH-UNDEF} \end{array}$$

These rules can be read as follows:

- Producing a value v_t and a memory m_t in the compiled program is justified by the possibility of producing a value v_s and a memory m_s in the source program such that $\Phi(v_t, v_s, m_t, m_s)$.
- Divergence in the compiled program is justified by the possibility of divergence in the source program.
- Any behavior b_t in the compiled program is justified by the possibility of undefined behavior in the source program.

We then define the *flexible behavioral refinement* between a target program T and a source program S :

$$T \leq S \triangleq \forall b_t, b_t \in T \implies \exists b_s, b_s \in S \wedge b_t \leq_{\text{beh}} b_s \quad (2)$$

This relation is similarly defined between configurations $t \leq s$. Whenever $T \leq S$ holds, every observable behavior produced by the compiled program T is validated by a matching behavior in the source program S with respect to the preorder $\leq_{\text{beh}}$.

1.4 Simulation Relations

While a refinement relation is formulated globally over whole-program behaviors, proving it directly over full execution traces is impractical. Compilers perform transformations whose correctness is best verified step by step. To bridge this gap, verified compilation relies on *simulation relations*, which maintain an invariant relating intermediate configurations of the source and target executions. Simulation relations are established in two flavors:

Backward simulation: each execution step taken by the target program must be matched by corresponding steps in the source program. Because it maps every target execution path back to an admissible source path, a backward simulation directly establishes behavioral refinement $t \sqsubseteq s$.

Forward simulation: each execution step taken by the source program is matched by steps in the target program. Forward simulations are more convenient to establish as compiler passes transform source constructs into target code, making inductive invariant proofs follow the structure of source reductions directly.

1.5 Coinductive Predicates

Proving a simulation relation requires relating execution steps of the target and source programs. When programs terminate, their execution consists of a finite sequence of reductions ending in a final configuration. Such finite behaviors are formalized using *inductive* predicates. However, programs that are designed to run indefinitely produce infinite reduction sequences that never encounter a terminal case. Establishing a simulation inductively for diverging executions is impossible, as it would require an infinite derivation tree, which inductive definitions forbid. To reason about infinite execution traces, we rely on *coinductive* predicates.

The presentation adopted below, which interprets coinduction in terms of infinite derivation trees, is an intuitive presentation adapted to our setting. For foundational treatments of coinduction, we refer the reader to Milner [Mil89], Sangiorgi [San12], Giménez [Gim98], and Pous [Pou16]. Throughout this report, we specify inductive and coinductive relations through inference rules:

- *Inductive rules* are denoted with a single horizontal bar:

$$\frac{H_1 \quad \dots \quad H_n}{\mathcal{I}}$$

Here, H_i are the premises and $\mathcal{I}$ is the inductive conclusion. A judgment holds inductively if and only if it can be established through a *finite derivation tree*, starting from base cases.

- *Coinductive rules* are denoted with a double horizontal bar:

$$\frac{H_1 \quad \dots \quad H_n}{\mathcal{C}}$$

Here, H_i are the premises and $\mathcal{C}$ is the coinductive conclusion. They generalize inductive rules to judgments admitting *potentially infinite derivation trees*.

These inference rules can be read in a goal-directed manner:

$$\frac{P \quad \text{Goal}'}{\text{Goal}}$$

To establish Goal, one validates the side conditions P and updates the proof goal to Goal' . When Goal' is an implication $H \implies \text{Goal}''$, the hypothesis H is assumed in the context to establish Goal'' . To combine inductive and coinductive reasoning within the same definition, a relation R is defined as the greatest fixed point¹ of an inductive operator. Inductive rules with a single bar require a finite proof tree for occurrences of R in the premises, while coinductive rules with a double bar permit infinite derivations.

For example, finite lists are defined inductively, requiring every list to terminate at nil. In contrast, infinite streams are defined coinductively, allowing endless constructor applications without base cases:

$$\frac{}{\text{nil} \in \text{List}(\mathbb{N})} \quad \frac{n \in \mathbb{N} \quad l \in \text{List}(\mathbb{N})}{\text{cons}(n, l) \in \text{List}(\mathbb{N})} \quad \frac{n \in \mathbb{N} \quad s \in \text{Stream}(\mathbb{N})}{\text{scons}(n, s) \in \text{Stream}(\mathbb{N})}$$

These distinct derivation structures translate into distinct proof principles:

- One reasons by *induction* when the *hypothesis* is an inductive judgment. The proof proceeds by case analysis on the constructors of the finite derivation. The induction hypothesis may only be applied to *strictly smaller subterms*, guaranteeing finite descent toward a base case.
- One reasons by *coinduction* when the *goal* to be established is a coinductive judgment. The proof constructs a potentially infinite derivation tree. The coinductive hypothesis refers to the goal itself, but it may only be invoked *underneath a constructor* (scons for streams). This condition, known as *productivity*, prohibits using the coinductive hypothesis to build circular proof trees.

A detailed example illustrating coinductive reasoning is provided in section A. The Rocq theorem prover [The26] supports coinductive predicates natively. In our formalization, we use the `coinduction` library developed by Pous [Pou26] to facilitate reasoning with coinductive simulation invariants.

1.6 Simulation Schemes & Stuttering

Equipped with coinductive predicates to handle potentially infinite executions, we can define a simulation relation to establish program refinements: $T \leq S$. In general, the source and target languages may differ: target configurations t evolve under $\rightarrow_t$ with finality test `is_finalt`, while source configurations s evolve under $\rightarrow_s$ with finality test `is_finals`. A *backward simulation* relates a target configuration t to a source configuration s , ensuring that every execution step of the target is matched by the source. Simulations are parameterized by a postcondition Φ , which specifies the invariant expected between the final values and memories when both programs terminate²

For terminating executions, the simulation requires that final configurations yield related values and memories. We formalize this condition using the predicate `both_finalΦ`. The assertion `both_finalΦ(t, s)` holds when both target and source configurations have reached final states, producing values v_t, v_s and memories m_t, m_s that satisfy $\Phi(v_t, v_s, m_t, m_s)$.

¹The relationship between greatest fixed points and coinduction follows from the Knaster–Tarski theorem; see Pous [Pou16] and Schäfer and Smolka [SS17] for comprehensive treatments.

²This corresponds to the relation Φ in the behavioral preorder $\leq_{\text{beh}}$.

This termination condition is expressed in the rule LOCK-FINAL in fig. 1. When the source program encounters undefined behavior, it enters a stuck state in a non-final configuration. In this case, LOCK-STUCK in fig. 1 immediately validates the simulation, leaving the target program free to perform arbitrary transitions or diverge. This asymmetry is allowed by the flexible refinement we want to prove.

For non-final configurations, the relation must ensure that the target state is capable of progressing and that every transition it performs corresponds to an execution step in the source program. Because executions may be non-terminating, this condition is expressed coinductively in rule LOCK-STEP in fig. 1. Under non-determinism, the target and source configurations are treated asymmetrically:

- The target program exhibits *demonic* non-determinism: the simulation must account for all reachable successor configurations t' .
- The source program exhibits *angelic* non-determinism: the proof only needs to exhibit the existence of a single matching successor s' .

This asymmetry matches the quantification structure of the direct refinement: we must account for every non-deterministic path chosen by the target and justify it with an admissible path in the source.

$$\begin{array}{c}
\frac{\text{both_final}_\Phi(t, s)}{t \sim_l s \{\Phi\}} \text{ LOCK-FINAL} \qquad \frac{\text{stuck}_s(s)}{t \sim_l s \{\Phi\}} \text{ LOCK-STUCK} \\
\\
\frac{\text{can_progress}_t(t) \quad \forall t', t \rightarrow_t t' \implies \exists s', s \rightarrow_s s' \wedge t' \sim_l s' \{\Phi\}}{t \sim_l s \{\Phi\}} \text{ LOCK-STEP}
\end{array}$$

Figure 1: Rules of the *lock-step simulation*.

The relation $\sim_l$ defined in fig. 1 is a *lock-step* backward simulation: the target and source programs advance at the exact same pace. While lock-step simulations suffice for simple transformations that preserve step counts, realistic compiler passes break this one-to-one correspondence. An optimization pass such as branch tunneling removes redundant jump instructions, causing the source program to execute steps that have no counterpart in the target. Conversely, lowering transformations decompose a single high-level source instruction into multiple low-level target instructions, requiring the target to execute several steps before the source advances.

To accommodate passes that alter step counts, one must allow *asynchronous* execution steps, where one program takes a step while the other remains stationary. Such a transition is called a *stuttering* step. A naive coinductive formulation might introduce unrestricted stuttering rules:

$$\frac{\text{can_progress}_t(t) \quad \forall t', t \rightarrow_t t' \implies t' \sim s}{t \sim s} \qquad \frac{s \rightarrow_s s' \quad t \sim s'}{t \sim s}$$

However, adding these unconstrained rules invalidates the soundness of the simulation relation: one cannot prove that $t \sim s \implies t \sqsubseteq s$. Without restrictions, a target program could perform infinitely many stuttering steps without the source program ever progressing. Under such a definition, an always diverging target program could be related to any source state. Conversely, a source program could perform infinitely many stuttering steps without the target program ever progressing. This would similarly allow relating any target program to a diverging source state.

To ensure soundness, the stuttering must remain finite. This is achieved by equipping the simulation with a well-founded measure $(W, \prec)$. The relation is indexed by a measure $i \in W$, written $t \sim_e^i s \{\Phi\}$. Whenever a stuttering step occurs, the index strictly decreases according to $\prec$. When both programs step simultaneously, progress is established and the measure may be freely reset.

Because stuttering is bounded by an explicit measure, this relation is called the *explicit simulation* [Cho+23]. The inference rules defining it are given in fig. 2. To simplify the notation for stuttering transitions, we write $t \sim_e^{\prec i} s \{\Phi\}$ as syntactic sugar for:

$$t \sim_e^{\prec i} s \{\Phi\} \triangleq \exists i', i' \prec i \wedge t \sim_e^{i'} s \{\Phi\}$$

The rules ESOURCESTEPS and ETARGETSTEPS govern asynchronous execution steps:

- In **ESOURCESTEPS**, the source program takes a step from s to s' while the target program stutters. Due to angelic non-determinism in the source, it suffices to exhibit a single matching successor s' with a strictly decreased measure $t \sim_e^i s' \{\Phi\}$.
- In **ETARGETSTEPS**, the target t must be able to make progress, and because target non-determinism is demonic, the simulation at a decreased measure $t' \sim_e^i s \{\Phi\}$ must be established for every transition $t \rightarrow_t t'$.

Since $(W, \prec)$ is well-founded, an infinite sequence of stuttering steps is impossible, guaranteeing that explicit simulation implies behavioral refinement. However, **EBOTHSTEPS** is the only rule that permits resetting the stuttering measure. This requires synchronizing the target and source steps that justify divergence, imposing a proof burden that we address in the following chapter.

$$\begin{array}{c}
\frac{\text{both_final}_\Phi(t, s)}{t \sim_e^i s \{\Phi\}} \text{ERELATED} \qquad \frac{\text{stuck}_s(s)}{t \sim_e^i s \{\Phi\}} \text{ESTUCK} \qquad \frac{s \rightarrow_s s' \quad t \sim_e^i s' \{\Phi\}}{t \sim_e^i s \{\Phi\}} \text{ESOURCESTEPS} \\
\\
\frac{\text{can_progress}_t(t) \quad \forall t', t \rightarrow_t t' \implies t' \sim_e^i s \{\Phi\}}{t \sim_e^i s \{\Phi\}} \text{ETARGETSTEPS} \\
\\
\frac{\text{can_progress}_t(t) \quad \forall t', t \rightarrow_t t' \implies \exists s', \exists i', s \rightarrow_s s' \wedge t' \sim_e^{i'} s' \{\Phi\}}{t \sim_e^i s \{\Phi\}} \text{EBOTHSTEPS}
\end{array}$$

Figure 2: Derivation rules of the *explicit simulation*.

2 The Free Simulation

While explicit simulations effectively prevent infinite stuttering, they impose a substantial proof burden. Every stuttering transition requires exhibiting a strictly decreasing measure in a well-founded order. Constructing such a measure can be difficult in practice, particularly when compiler optimizations rearrange loop structures. In such scenarios, finding a global metric that strictly decreases on every asynchronous step while remaining invariant across complex control transfers is often impractical.

To overcome this rigidity, we adapt the *Stuttering for Free* framework introduced by Cho et al. [Cho+23]. We extend their foundational technique to accommodate skewed simulation relations. All underlying theoretical results have been formalized and verified within our Rocq development.

2.1 Implicit Simulation

An effective approach to eliminating explicit stuttering measures is to combine inductive and coinductive reasoning within a single relation. In this formulation, finite sequences of asynchronous execution steps are justified inductively, whereas mutual progress steps are justified coinductively. Because explicit stuttering indices are entirely omitted from the relation, this family of relations is referred to as *implicit simulations* by Cho et al. [Cho+23]. We define an implicit simulation relation, denoted $t \lesssim s \{\Phi\}$, whose inference rules are presented in fig. 3.

The rule **IRELATED** characterizes terminating executions, mirroring the **ERELATED** rule of the explicit simulation presented in fig. 2. Stuttering is soundly handled without requiring an explicit decreasing measure. Because **ISOURCESTEPS** and **ITARGETSTEPS** are inductive constructors, they permit only finitely many consecutive asynchronous transitions. The coinductive constructor **IBOTHSTEPS** requires both programs to execute a synchronized step, removing the inductive proof obligation and enabling unbounded mutual progress.

2.2 Step-Indexed Free Simulation

While the implicit simulation eliminates explicit stuttering measures, it remains restrictive: the coinductive rule **IBOTHSTEPS** demands that both programs take a step at the exact moment when coinductive reasoning is resumed. Removing this lock-step synchronization requirement leads to simpler invariants and more direct verification proofs [Cho+23]. This decoupling is the core idea developed in the *Stuttering for Free* framework.

$$\begin{array}{c}
\frac{\text{both_final}_\Phi(t, s)}{t \lesssim s \{\Phi\}} \text{IRELATED} \qquad \frac{\text{stuck}_s(s)}{t \lesssim s \{\Phi\}} \text{ISOURCESTUCK} \\
\\
\frac{s \twoheadrightarrow_s s' \quad t \lesssim s' \{\Phi\}}{t \lesssim s \{\Phi\}} \text{ISOURCESTEPS} \\
\\
\frac{\text{can_progress}_t(t) \quad \forall t', t \twoheadrightarrow_t t' \implies t' \lesssim s \{\Phi\}}{t \lesssim s \{\Phi\}} \text{ITARGETSTEPS} \\
\\
\frac{\text{can_progress}_t(t) \quad \forall t', t \twoheadrightarrow_t t' \implies \exists s', s \twoheadrightarrow_s s' \wedge t' \lesssim s' \{\Phi\}}{t \lesssim s \{\Phi\}} \text{IBOTHSTEPS}
\end{array}$$

Figure 3: Derivation rules of the *skewed implicit simulation*.

The *free simulation*, denoted $t \lesssim_i s \{\Phi\}$, overcomes this limitation by decoupling execution steps from coinductive progress. The relation is parameterized by two well-founded index domains, $(J, \prec_J)$ for the target side and $(I, \prec_I)$ for the source side. These domains track the progress accumulated by each side since the previous coinductive step. The formal inference rules of the skewed free simulation are presented in fig. 4. Like the implicit simulation, it accommodates arbitrary postconditions Φ , and uses the rule FSOURCESTUCK to exploit any undefined behavior in the source program.

$$\begin{array}{c}
\frac{\text{both_final}_\Phi(t, s)}{t \lesssim_i s \{\Phi\}} \text{FRELATED} \qquad \frac{\text{stuck}_s(s)}{t \lesssim_i s \{\Phi\}} \text{FSOURCESTUCK} \\
\\
\frac{s \twoheadrightarrow_s s' \quad t \lesssim_{i'} s' \{\Phi\}}{t \lesssim_i s \{\Phi\}} \text{FSOURCESTEPS} \\
\\
\frac{\text{can_progress}_t(t) \quad \forall t', t \twoheadrightarrow_t t' \implies \exists j', t' \lesssim_{j'} s \{\Phi\}}{t \lesssim_i s \{\Phi\}} \text{FTARGETSTEPS} \\
\\
\frac{j' \prec_J j \quad i' \prec_I i \quad t \lesssim_{i'} s \{\Phi\}}{t \lesssim_i s \{\Phi\}} \text{FPROGRESS}
\end{array}$$

Figure 4: Derivation rules of the *skewed free simulation*.

The design of the free simulation relies on a fundamental shift in perspective regarding indices. In an explicit simulation, an index represents an upper bound on *future* stuttering steps. In the free simulation, indices instead record *past* progress accumulated since the previous coinductive step. Consequently, a simulation statement with smaller indices is stronger than one with larger indices. Indices are meaningful primarily during the coinductive proof of the simulation. Once the simulation is established, it holds independently of the chosen indices, a property we justify formally later in theorem 2.

The source and target programs take transitions asynchronously via the inductive rules FTARGETSTEPS and FSOURCESTEPS. Whenever a program takes a step, its corresponding step index is updated (j' or i'). Importantly, j' and i' are not required to be immediate successors of j and i ; instead, one is free to select any new index, whether smaller or larger.

This flexibility does not compromise soundness. The only rule that enables coinductive reasoning is FPROGRESS, which requires both indices to strictly decrease, from j down to $j' \prec_J j$ and from i down to $i' \prec_I i$. Because the relations $\prec_J$ and $\prec_I$ are well-founded, establishing a simulation between non-terminating executions demands that FPROGRESS be applied infinitely often. This, in turn, requires indices to be repeatedly increased through FTARGETSTEPS and FSOURCESTEPS. This structure decouples stepping from coinductive synchronization, allowing multiple asynchronous transitions on either side to be matched flexibly.

Moreover, coinductive reasoning over the free simulation is governed by the following principle:

Lemma 2 (Coinduction Principle for Free Simulation). To prove that $t \lesssim_i s \{\Phi\}$, one may assume the coinductive hypothesis for strictly larger indices:

$$\forall j' \succ_J j, \forall i' \succ_I i, \quad t \lesssim_{i'} s \{\Phi\}.$$

Intuitively, this principle allows one to establish that two programs simulate each other by assuming that the simulation relation already holds for the current states whenever both sides have made strict progress. A concrete example illustrating step-indexed reasoning in the free simulation is presented in section B.

2.3 Properties

As observed earlier, proving a free simulation for smaller indices yields a stronger statement that implies the simulation for larger indices. This monotonicity property is useful during coinductive proofs. Formally:

Lemma 3 (Monotonicity of Free Simulation). If $t_{j \lesssim_i} s \{\Phi\}$ holds, then for all indices $j \prec_J j'$ and $i \prec_I i'$, and any weaker postcondition $\Phi \implies \Phi'$, the simulation $t_{j' \lesssim_{i'}} s \{\Phi'\}$ holds.

The free simulation is sound with respect to skewed behavioral refinement:

Theorem 1 (Soundness of Free Simulation). Let P_t and P_s be programs, with initial states t and s , respectively. If $t_{j \lesssim_i} s \{=\}$ holds for some indices j and i , then $P_t \leq P_s$.

In addition to soundness, we established the formal equivalence among the three simulation frameworks presented: explicit simulation (fig. 2), implicit simulation (fig. 3), and free simulation (fig. 4). This equivalence is established through a chain of cyclic implications:

$$\text{Explicit Simulation} \xRightarrow{\textcircled{1}} \text{Implicit Simulation} \xRightarrow{\textcircled{2}} \text{Free Simulation} \xRightarrow{\textcircled{3}} \text{Explicit Simulation}$$

The implications $\textcircled{1}$ and $\textcircled{2}$ proceed directly by coinduction. The final implication $\textcircled{3}$ is the most intricate: it requires synthesizing a well-founded stuttering measure from the derivation tree of the free simulation. A direct consequence of this equivalence is the index irrelevance theorem:

Theorem 2 (Index Irrelevance). Let $(J, \prec_J)$ and $(I, \prec_I)$ be well-founded relations without isolated elements³. If there exist indices $j \in J$ and $i \in I$ such that $t_{j \lesssim_i} s \{\Phi\}$, then for any other well-founded structures $(J', \prec_{J'})$ and $(I', \prec_{I'})$, and any indices $j' \in J'$ and $i' \in I'$, the relation $t_{j' \lesssim_{i'}} s \{\Phi\}$ holds.

This theorem cannot be applied within a proof by coinduction⁴. This theorem highlights a fundamental aspect of step indices in the free simulation: they carry no semantic weight once the simulation relation is established. Their role is purely to ease the proof of the simulation. They are used to synchronize transitions and bound asynchronous stuttering.

Our statement refines the index irrelevance theorem formulated by Cho et al. [Cho+23]. Their original formulation, phrased in terms of ordinals, implies that well-founded structures contain no isolated elements. However, isolated elements could cause FPROGRESS transitions to be uninhabited, preventing the equivalence. This requirement was identified during our Rocq formalization of the metatheory of the free simulation.

This theorem implies that any free simulation derivation can be carried out using the two-element boolean order $\mathbb{B} = \{\text{false} \prec \text{true}\}$:

$$(\exists j \in J, i \in I, t_{j \lesssim_i} s \{\Phi\}) \iff (\exists j, i \in \mathbb{B}, t_{j \lesssim_i} s \{\Phi\}).$$

Under the boolean order, false indicates that a program has not yet taken a step, whereas true signals that progress has occurred. In principle, this shows that free simulation proofs do not require tracking exact step counts, but only whether progress has occurred since the previous coinductive step. Nevertheless, in the remainder of our work, we adopt the domain $(\mathbb{N}, <)$ as our index domain. Indeed, indices in $\mathbb{N}$ correspond directly to the number of steps executed during symbolic evaluation, providing clear intuition that facilitates interactive verification in Rocq.

3 The Register Transfer Language (μ RTL)

To evaluate our framework on realistic compiler intermediate representations, we formalize a low-level imperative language, μ RTL, based on the Register Transfer Language (RTL) of CompCert [Ler09b]. Storage in μ RTL is organized into two distinct mechanisms. Each function invocation operates on its own local register bank, which holds the values of its local variables. During execution, the language maintains a call stack of activation frames, preserving caller contexts. In addition, functions share a global heap memory.

³An element is isolated if it has no predecessors or successors in the relation.

⁴Formally, when free simulation is characterized as a greatest fixed point, monotonicity (lemma 3) holds for all intermediate post-fixpoints of the coinductive proofs. In contrast, index irrelevance (theorem 2) is a property of the greatest fixed point itself, and thus cannot be assumed as a hypothesis within an unclosed coinductive derivation.

3.1 Values, Registers, and Memory

Computations in μ RTL manipulate first-order values. There are four kinds of values, where loc represents allocated heap locations:

$$v \in \text{val} ::= \text{VInt}(z) \mid \text{VBool}(b) \mid \text{VPtr}(l) \mid \text{VUndef} \\ \text{with } z \in \mathbb{Z}, b \in \{\text{true}, \text{false}\}, \text{ and } l \in \text{loc}$$

VUndef represents an undefined value, used to model uninitialized data. It can be copied between registers, stored into memory cells, or passed as function arguments. However, attempting to execute an instruction that requires a specific kind of value, such as an arithmetic calculation, will cause the execution to get stuck.

To relate source and target values, we define a similarity predicate $v_t \bowtie_I v_s$. This predicate is parameterized by a binary relation on locations, $I \subseteq \text{loc} \times \text{loc}$. This relation will be made precise later in section 4.2. The deduction rules of the value similarity predicate are given in fig. 5. We note that the rule VAL-UNDEF is asymmetric: whenever the source program produces an undefined value VUndef , it can be related to any value v_t produced by the target program. This asymmetry allows the compiler to replace undefined source computations with more defined target operations.

$$\begin{array}{c} \frac{z_t = z_s}{\text{VInt}(z_t) \bowtie_I \text{VInt}(z_s)} \quad \text{VAL-INT} \qquad \frac{b_t = b_s}{\text{VBool}(b_t) \bowtie_I \text{VBool}(b_s)} \quad \text{VAL-BOOL} \\[10pt] \frac{(l_t, l_s) \in I}{\text{VPtr}(l_t) \bowtie_I \text{VPtr}(l_s)} \quad \text{VAL-PTR} \qquad \frac{}{v_t \bowtie_I \text{VUndef}} \quad \text{VAL-UNDEF} \end{array}$$

Figure 5: The value similarity predicate.

In μ RTL, registers store values that can be actively manipulated in computations. They are referenced by an identifier $r \in \text{reg}$. μ RTL functions can use as many registers as they require. A μ RTL program can also interact with the heap. The heap is indexed by locations $l \in \text{loc}$, and the program can read, write, allocate, and free memory cells associated with locations.

3.2 Syntax and Program Structure

High-level imperative languages employ *structured* control-flow constructs, such as nested conditionals, while loops, and compound statements. In structured languages, execution paths follow the hierarchical source code structure of the program. In contrast, μ RTL is an *unstructured* language. Control flow in μ RTL is represented as an explicit directed graph whose nodes $pc \in \text{node}$ represent program points. Each instruction explicitly specifies the program points of its successors, eliminating the need for loop or block constructs. The instruction set and operator grammar of μ RTL are defined in fig. 6. A μ RTL function f is a record containing:

$$f \in \text{rtl_function} ::= \left\{ \begin{array}{ll} \text{fn_name} \in \text{ident} & \text{function identifier} \\ \text{fn_regs} \in \text{list reg} & \text{parameter list} \\ \text{fn_entrypoint} \in \text{node} & \text{function entry point} \\ \text{fn_code} \in \text{node} \rightarrow \text{instr} & \text{control-flow graph} \end{array} \right\}$$

We enforce the invariant that input parameter registers contain no duplicates. The code map fn_code is a finite map associating program points with instructions. In the following, we write $f[pc] = \langle i \rangle$ to assert that the code map of f contains instruction i at program point pc . A μ RTL program P consists of a list of functions with a distinguished entry function. We require that function names in the program are unique, and that the entry function is present.

3.3 Small-Step Operational Semantics

The execution of a μ RTL program is formalized as a small-step operational transition system. The configurations are tuples (σ, z, m) , where σ is a call stack, $m \in \text{memory}$ is the global heap, and z is a control state that can take three distinct forms:

- State (f, pc, ρ) is the main state. It refers to an execution occurring inside function f at program point pc , with local register bank ρ .
- CallState $(f, \vec{v})$ represents the beginning of the invocation of function f with argument values $\vec{v}$.
- ReturnState (v) represents the completion of a function execution with return value v .

$i \in \text{instr}$	$::=$	$\text{nop} \rightarrow pc$	unconditional jump
		$dst := \text{op } \vec{r} \rightarrow pc$	pure arithmetic or logical operation
		$dst := !src \rightarrow pc$	heap memory load
		$!addr := src \rightarrow pc$	heap memory store
		$dst := \text{alloc}() \rightarrow pc$	heap cell allocation
		$\text{free } addr \rightarrow pc$	heap cell deallocation
		$\text{if } cond \text{ then } pc_{\text{true}} \text{ else } pc_{\text{false}}$	conditional branching
		$dst := \text{call } fn(\vec{r}) \rightarrow pc$	function invocation
		$\text{ret } src$	function return
op	$::=$	$\text{Add} \mid \text{Sub} \mid \text{Mul} \mid \text{Div}$	integer arithmetic
		$\text{Incr} \mid \text{Decr}$	increment and decrement
		$\text{ImmInt}(z) \mid \text{ImmBool}(b)$	constant immediate values
		$\text{Move} \mid \text{EqZ}$	register copy and zero test

where $dst, src, addr, cond \in \text{reg}$, $\vec{r} \in \text{list reg}$, $fn \in \text{ident}$, and $pc, pc_{\text{true}}, pc_{\text{false}} \in \text{node}$.

Figure 6: The syntax of μRTL instructions and operators.

The call stack σ is a list of stack frames. Each stack frame $\text{Stackframe}(dst, f, pc, \rho)$ stores the caller context: the destination register dst designated to receive the return value, the caller function f , the return program point pc , and the caller register bank ρ . Structuring transitions through explicit $\text{CallState}(f, \vec{v})$ and $\text{ReturnState}(v)$ configurations cleanly decouples caller and callee environments. In CompCert [Ler09b], this abstraction is also used to integrate external function calls that jump directly from a call state to a return state.

A register bank and the memory heap are formalized as shown in fig. 7. By explicitly recording Freed cells rather than removing locations from the domain of m , our memory model distinguishes dangling pointer dereferences and double frees from accesses to unallocated memory.

$\rho \in \text{regbank}$	$::=$	$\text{reg} \rightarrow \text{val}$	register bank
$m \in \text{memory}$	$::=$	$\text{loc} \rightarrow \text{cell}$	memory heap
$c \in \text{cell}$	$::=$	$\text{Allocated}(v) \mid \text{Freed}$	heap memory cell

Notation	Operation Semantics
$\rho[r]$	Read register r ; yields VUndef if $r \notin \text{dom}(\rho)$
$\rho[\vec{r}]$	Pointwise extension to register lists, returning $\vec{v}$
$\rho[r \mapsto v]$	Register assignment updating r to value v
$m[l]$	Read cell at location l
$m[l \mapsto \text{Allocated}(v)]$	Update allocated cell at l to value v
$m \uplus \{l \mapsto \text{Allocated}(v)\}$	Allocate fresh cell $l \notin \text{dom}(m)$ initialized with value v
$m[l \mapsto \text{Freed}]$	Deallocate active cell at l , setting its status to Freed

Figure 7: Formalization of register banks and memory heap operations.

The step relation $P \twoheadrightarrow ss'$ is defined by inference rules, selected rules of which are presented in fig. 8. Execution of a function progresses through three main phases ($\text{CallState} \rightarrow \text{State} \rightarrow \text{ReturnState}$):

1. In the beginning, the state is a call state $\text{CallState}(f, \vec{v})$. Rule STEP-ENTRY steps into the body of the function at its entry point. This initializes the register bank with the values given for the input registers of f . The resulting state is an active state $\text{State}(f, pc_0, \rho)$.
2. Instruction rules then execute the body of the function. These rules apply only when they correspond to the next instruction to be executed, referenced by pc in f . The evaluation of some instructions is partial: in the STEP-OP rule, if $\text{eval_op}(\text{op}, \vec{v})$ returns None , the execution is stuck.
3. When the function reaches a return instruction, rule STEP-RET applies. It evaluates its return register and enters a return state $\text{ReturnState}(v)$.

$$\begin{array}{c}
\frac{f[pc] = \langle dst := op \vec{r} \rightarrow pc' \rangle \quad \rho[\vec{r}] = \vec{v} \quad \text{eval_op}(op, \vec{v}) = \text{Some}(v)}{P \twoheadrightarrow (\sigma, \text{State}(f, pc, \rho), m)(\sigma, \text{State}(f, pc', \rho[dst \mapsto v]), m)} \quad \text{STEP-OP} \\
\\
\frac{f[pc] = \langle dst := \text{alloc}() \rightarrow pc' \rangle \quad l \notin \text{dom}(m) \quad v \in \text{val}}{P \twoheadrightarrow (\sigma, \text{State}(f, pc, \rho), m)(\sigma, \text{State}(f, pc', \rho[dst \mapsto \text{VPtr}(l)]), m \uplus \{l \mapsto \text{Allocated}(v)\})} \quad \text{STEP-ALLOC} \\
\\
\frac{f[pc] = \langle \text{free } addr \rightarrow pc' \rangle \quad \rho[addr] = \text{VPtr}(l) \quad m[l] = \text{Allocated}(_) }{P \twoheadrightarrow (\sigma, \text{State}(f, pc, \rho), m)(\sigma, \text{State}(f, pc', \rho), m[l \mapsto \text{Freed}])} \quad \text{STEP-FREE} \\
\\
\frac{f[pc] = \langle dst := \text{call } fn(\vec{r}) \rightarrow pc' \rangle \quad \text{find_fun}(P, fn) = \text{Some}(f_{\text{callee}}) \quad \rho[\vec{r}] = \vec{v}}{P \twoheadrightarrow (\sigma, \text{State}(f, pc, \rho), m)(\text{Stackframe}(dst, f, pc', \rho) :: \sigma, \text{CallState}(f_{\text{callee}}, \vec{v}), m)} \quad \text{STEP-CALL} \\
\\
\frac{|\vec{v}| = |\text{fn_regs}(f)| \quad \rho = \text{init_regs}(\text{fn_regs}(f), \vec{v})}{P \twoheadrightarrow (\sigma, \text{CallState}(f, \vec{v}), m)(\sigma, \text{State}(f, \text{fn_entrypoint}(f), \rho), m)} \quad \text{STEP-ENTRY} \\
\\
\frac{f[pc] = \langle \text{ret } src \rangle \quad \rho[src] = v}{P \twoheadrightarrow (\sigma, \text{State}(f, pc, \rho), m)(\sigma, \text{ReturnState}(v), m)} \quad \text{STEP-RET} \\
\\
\frac{}{P \twoheadrightarrow (\text{Stackframe}(dst, f, pc', \rho) :: \sigma, \text{ReturnState}(v), m)(\sigma, \text{State}(f, pc', \rho[dst \mapsto v]), m)} \quad \text{STEP-RETURN}
\end{array}$$

Figure 8: Selected rules of the small-step operational semantics of μRTL .

4. If the call stack is not empty, the caller resumes execution via STEP-RETURN, popping the top stack frame, writing the return value into register dst , and branching to the next pc .
5. When a function call occurs, rule STEP-CALL applies. The caller evaluates the argument registers, pushes a fresh stack frame recording its continuation point pc' and register bank ρ , and transitions to a call state $\text{CallState}(f_{\text{callee}}, \vec{v})$.

An execution state is final when the call stack is empty and the program is in a return state. The final value is the one stored in the control state.

3.4 Stack Modularity and Execution Invariants

Reasoning directly about executions with arbitrary call stacks introduces unnecessary proof complexity. Without stack modularity, one would need to maintain an explicit invariant over caller stack frames and prove that it is preserved across every step. To achieve modularity, we prove framing lemmas directly for the operational semantics. These lemmas establish that any execution step derived in isolation with an empty call stack remains valid when embedded into an arbitrary caller stack context. When verifying a function call, the callee is thus verified starting from an empty call stack $([], \text{CallState}(f, \vec{v}), m)$. Once the callee reduces to $([], \text{ReturnState}(v), m')$, the execution is lifted back into the caller's stack context, reconnecting the callee's execution with the caller's continuation.

4 Relational Separation Logic

Having defined our free simulation framework, we require a program logic to specify and verify relational properties of program executions. Reasoning about memory transformations using global heap predicates imposes a heavy verification burden, as every intermediate step must maintain global invariants. To enable local reasoning over pairs of memory heaps, we develop a relational separation logic.

4.1 Separation Logic

Separation logic [Rey02; Cha20] provides a modular foundation for reasoning about programs that manipulate memory. In classical Hoare logic, assertions describe properties of the entire heap. Establishing that an operation modifies only a restricted memory region requires explicitly proving that untouched memory regions remain unchanged. Separation logic

solves this challenge by introducing the separating conjunction $P * Q$, which asserts that the memory can be split into two disjoint sub-heaps satisfying P and Q , respectively.

In verified compilation, we reason relationally about two program executions: the target program and the source program. Because each execution operates over its own memory heap, we formulate a relational separation logic [Yan07] whose assertions describe pairs of memory states (m_t, m_s) .

In modern interactive proof assistants, the Iris framework [Jun+18; Kre+17] serves as the standard platform for mechanized separation logic in Rocq. Iris provides expressive mechanisms for higher-order concurrent reasoning, including step-indexed models, higher-order ghost state, and user-defined resource algebras. In our setting, these advanced features are not yet needed. Instead, we formalize our relational separation logic directly as a shallow embedding in Rocq. We define the type of relational propositions, denoted rProp , as a binary relation over memories:

$$\text{rProp} \triangleq \text{memory} \rightarrow \text{memory} \rightarrow \text{Prop}$$

By convention, the first argument denotes the target memory m_t , and the second argument denotes the source memory m_s . An assertion $P \in \text{rProp}$ represents a predicate that expresses ownership over the memory footprints of both programs, specifying the contents of target and source sub-heaps.

Connective	Semantics	Description
$\ulcorner \psi \urcorner$	$\lambda m_t m_s. m_t = \emptyset \wedge m_s = \emptyset \wedge \psi$	Pure assertion
emp	$\lambda m_t m_s. m_t = \emptyset \wedge m_s = \emptyset$	Empty memory heaps
$\forall x, P(x)$	$\lambda m_t m_s. \forall x, P(x)(m_t, m_s)$	Universal quantification
$\exists x, P(x)$	$\lambda m_t m_s. \exists x, P(x)(m_t, m_s)$	Existential quantification
$P * Q$	$\lambda m_t m_s. \exists m_t^P m_t^Q m_s^P m_s^Q, m_t = m_t^P \uplus m_t^Q \wedge m_s = m_s^P \uplus m_s^Q$ $\wedge P(m_t^P, m_s^P) \wedge Q(m_t^Q, m_s^Q)$	Separating conjunction
$P \multimap Q$	$\lambda m_t m_s. \forall m'_t m'_s, P(m'_t, m'_s) \implies Q(m_t \uplus m'_t, m_s \uplus m'_s)$	Magic wand
$\Box P$	$\lambda m_t m_s. P(\emptyset, \emptyset)$	Persistent modality
$P \vdash Q$	$\forall m_t m_s, P(m_t, m_s) \implies Q(m_t, m_s)$	Logical entailment
$P \dashv\vdash Q$	$(P \vdash Q) \wedge (Q \vdash P)$	Logical equivalence

Figure 9: Base connectives of our relational separation logic.

The semantic definitions of our base logical connectives are summarized in fig. 9. The entailment $P \vdash Q$ asserts that whenever P holds on a memory pair, Q holds as well. Logical equivalence $P \dashv\vdash Q$ holds when both directions of entailment hold. The connectives satisfy the standard algebraic properties of separation logic:

- The separating conjunction $P * Q$ splits both the target heap m_t and the source heap m_s into disjoint sub-heaps⁵, assigning one part to P and the other part to Q . This operator enforces unique ownership of resources.
- The persistent modality $\Box P$ asserts that P holds on empty heaps. Because a persistent assertion requires no memory resources, it represents a duplicable fact: $\Box P \vdash \Box P * \Box P$.

Our memory logic is *linear* rather than affine. In an affine logic, heap chunks can be silently discarded via weakening: $P * Q \vdash P$. In our linear formulation, proving $P \vdash Q$ guarantees that every memory cell described by P is accounted for in Q . This strict resource discipline prevents unnoticed resource leaks.

To preserve the interactive proof engineering benefits of the Iris Proof Mode (MoSeL) [Kre+18], we prove that our connectives satisfy the required typeclass hierarchy. This allows our formalization to use standard Iris tactics while executing directly on our shallow relational model.

4.2 Relational Assertions & Memory Predicates

Building on our memory model (fig. 7), we introduce primitive memory assertions for target and source states. Relational points-to and freed assertions are defined in fig. 10. The predicate $l \rightarrow_t v$ asserts that location l in the target heap contains value v , while requiring the source heap to be empty. Symmetrically, $l \rightarrow_s v$ asserts ownership of an allocated cell in the source heap. The predicates $\text{free}_t(l)$ and $\text{free}_s(l)$ assert ownership of deallocated memory cells in the target and source heaps, respectively.

⁵Here $\uplus$ denotes the disjoint union of memory heaps, which is defined only when its operand domains are disjoint.

$$\begin{aligned}
l \rightarrow_t v &\triangleq \lambda m_t m_s. m_t = \{l \mapsto \text{Allocated}(v)\} \wedge m_s = \emptyset \\
l \rightarrow_s v &\triangleq \lambda m_t m_s. m_t = \emptyset \wedge m_s = \{l \mapsto \text{Allocated}(v)\} \\
\text{free}_t(l) &\triangleq \lambda m_t m_s. m_t = \{l \mapsto \text{Freed}\} \wedge m_s = \emptyset \\
\text{free}_s(l) &\triangleq \lambda m_t m_s. m_t = \emptyset \wedge m_s = \{l \mapsto \text{Freed}\}
\end{aligned}$$

Figure 10: Primitive relational points-to and free assertions.

Memory allocations in the target and source programs do not necessarily yield identical physical addresses. To account for such address variations, we maintain an address mapping $I \subseteq \text{loc} \times \text{loc}$, where each pair $(l_t, l_s) \in I$ relates a target location l_t to a source location l_s . We enforce that I is a bijective mapping: target locations are uniquely related to source locations. This mapping I parameterizes the value similarity relation $v_t \bowtie_I v_s$ (fig. 5). Allocating fresh memory in target or source does not extend I immediately. Instead, lemma 5 allows adding the fresh address pair to I at convenience. Because the mapping grows monotonically, existing value relations are preserved:

Lemma 4 (Monotonicity of $\bowtie_I$). If $I \subseteq I'$, then $v_t \bowtie_I v_s \implies v_t \bowtie_{I'} v_s$.

To relate target and source heaps across program executions, we draw upon the principles introduced by Gähler et al. [Gäh+22] in *Simuliris*, and define the memory injection predicate $\text{meminj}_E(I)$. The parameter $E \subseteq I$ designates a set of active exceptions, which is empty by default. The predicate $\text{meminj}_\emptyset(I)$ asserts collective ownership over all memory cells registered in I . Specifically, it asserts that every allocated pair $(l_t, l_s) \in I$ holds similar values ($v_t \bowtie_I v_s$), and that deallocated cells are consistently marked as freed in both heaps. When fresh cells are allocated, the injection can be extended with the new address pair:

Lemma 5 (Injection Extension). When allocating fresh memory cells in target and source, the newly allocated pair (l_t, l_s) can be added to the injection I provided that the stored values satisfy $v_t \bowtie_I v_s$.

Because $\text{meminj}_E(I)$ retains ownership of every cell in $I \setminus E$, verifying an instruction that reads, writes, or deallocates a specific location pair $(l_t, l_s) \in I$ requires accessing its underlying points-to or freed assertion. Removing (l_t, l_s) directly from the mapping I to obtain this assertion would shrink the domain of I , breaking monotonicity and invalidating the similarity of existing pointers. Instead, we temporarily add (l_t, l_s) to the exception set E , which extracts individual ownership without altering the bijection I .

This extraction and subsequent reintegration of memory assertions are governed by two complementary lemmas:

Lemma 6 (Injection Extraction). When $(l_t, l_s) \in I \setminus E$, the assertion $\text{meminj}_E(I)$ can be opened to extract ownership of the points-to or freed assertion for (l_t, l_s) , updating the exception set to $E \cup \{(l_t, l_s)\}$.

Lemma 7 (Injection Reintegration). Once a memory operation completes, the updated points-to assertion $l_t \rightarrow_t v_t * l_s \rightarrow_s v_s$ (satisfying $v_t \bowtie_I v_s$) or freed assertion $\text{free}_t(l_t) * \text{free}_s(l_s)$ is merged back into the injection, restoring the exception set to $E \setminus \{(l_t, l_s)\}$.

These lemmas will be used to temporarily extract points-to assertions for a location pair $(l_t, l_s) \in I$ when the source performs a memory operation. The exposed pair can then be used to justify a memory step in the target. Once both operations are completed, the updated assertions are reintegrated back into the memory injection.

4.3 Relational Hoare Quadruples

By combining the free simulation with relational separation logic, we define relational Hoare quadruples. For fixed target and source programs P_t and P_s , a relational Hoare quadruple relates a target function f_t to a source function f_s . The formal definition is given by:

$$\{P\} f_t \lesssim f_s \{Q\} \triangleq \Box \left(\forall \vec{v}_t \vec{v}_s. P(\vec{v}_t, \vec{v}_s) * ([], \text{CallState}(f_t, \vec{v}_t)) \text{ } 0 \lesssim_0 ([], \text{CallState}(f_s, \vec{v}_s)) \{Q\} \right)$$

where P is a precondition relating the target and source argument vectors, and Q is a postcondition specifying the returned values. The simulation is lifted to an assertion in rProp :

$$t \text{ } j \lesssim_i s \{ \Phi \} \triangleq \lambda m_t m_s. (t, m_t) \text{ } j \lesssim_i (s, m_s) \{ \Phi \}$$

The semantics of this quadruple is directly inherited from the free simulation: whenever the initial function arguments and heap states satisfy $P(\vec{v}_t, \vec{v}_s)$, if the source execution terminates without encountering undefined behavior, then the target execution also terminates with return values and memories satisfying $Q(v_t, v_s)$. If the source execution diverges, the target execution diverges as well. If the source execution encounters a stuck state, the simulation relation grants the compiler complete freedom.

The definition of the Hoare quadruple is guarded by the persistent modality $\Box$ to guarantee that the specification holds independently of heap ownership. As a consequence, a verified quadruple constitutes a duplicable resource that can be reused across multiple call sites throughout the program.

5 μ RTL Proof Rules

Having established a simulation relation, the semantics of μ RTL, and a separation logic over pairs of memories, we combine these components into a relational proof system. To facilitate the verification of code transformations, we establish a structured collection of relational proof rules operating directly on the simulation judgment $t \text{ }_j \lesssim_i s \{ \Phi \}$. These rules are all proved sound with respect to the simulation relation and the μ RTL semantics.

5.1 Asynchronous Rules

Our simulation relation decouples the execution of the target program from that of the source program. This separation yields asynchronous proof rules that advance one program while keeping the configuration of the other fixed.

Target proof rules express the conditions under which the target program can execute an instruction. The rules governing target execution of loads, allocations, and operations are displayed in fig. 11. The source stepping rules mirror the target rules, operating on the source program counter, source registers, and source memory assertions.

$$\begin{array}{c}
 \text{eval_op}(\text{op}, \vec{v}) = \text{Some}(v) \quad \rho[\vec{r}] = \vec{v} \\
 \frac{f[pc] = \langle dst := \text{op } \vec{r} \rightarrow pc' \rangle \quad (\sigma, \text{State}(f, pc', \rho[dst \mapsto v])) \text{ }_{j'} \lesssim_i c \{ \Phi \}}{(\sigma, \text{State}(f, pc, \rho)) \text{ }_j \lesssim_i c \{ \Phi \}} \quad \text{TARGET-OP} \\
 \\
 \frac{f[pc] = \langle dst := !src \rightarrow pc' \rangle \quad \rho[src] = \text{VPtr}(l) \quad l \rightarrow_t v \quad l \rightarrow_t v * (\sigma, \text{State}(f, pc', \rho[dst \mapsto v])) \text{ }_{j'} \lesssim_i c \{ \Phi \}}{(\sigma, \text{State}(f, pc, \rho)) \text{ }_j \lesssim_i c \{ \Phi \}} \quad \text{TARGET-LOAD} \\
 \\
 \frac{f[pc] = \langle dst := \text{alloc}() \rightarrow pc' \rangle \quad \forall l, v, l \rightarrow_t v * (\sigma, \text{State}(f, pc', \rho[dst \mapsto \text{VPtr}(l)])) \text{ }_{j'} \lesssim_i c \{ \Phi \}}{(\sigma, \text{State}(f, pc, \rho)) \text{ }_j \lesssim_i c \{ \Phi \}} \quad \text{TARGET-ALLOC}
 \end{array}$$

Figure 11: Asynchronous rules for allocation and operation instructions in the target program.

A key semantic difference between target and source execution appears in the allocation rule TARGET-ALLOC versus SOURCE-ALLOC. In our operational semantics, allocating a fresh memory cell non-deterministically initializes its content with an arbitrary value. Because the target is demonic, the continuation in TARGET-ALLOC must hold universally for *all* possible fresh locations l and all initial values v . In contrast, because the source is angelic, when applying SOURCE-ALLOC, the value of the allocated memory cell may be chosen existentially:

$$\frac{v \in \text{val} \quad f[pc] = \langle dst := \text{alloc}() \rightarrow pc' \rangle \quad \forall l, l \rightarrow_s v * c \text{ }_{j'} \lesssim_{i'} (\sigma, \text{State}(f, pc', \rho[dst \mapsto \text{VPtr}(l)])) \{ \Phi \}}{c \text{ }_j \lesssim_i (\sigma, \text{State}(f, pc, \rho)) \{ \Phi \}} \quad \text{SOURCE-ALLOC}$$

5.2 Undefined Behavior Exploitation

By the FSOURCESTUCK rule, our free simulation is skewed. When the source program reaches a stuck configuration, the simulation property $t \text{ }_j \lesssim_i s \{ \Phi \}$ holds directly. This property provides a powerful proof technique: whenever the source program is about to step, we can safely assume that the configuration is not stuck. Otherwise the proof obligation would

be instantly discharged. For instance, the exploit rule SOURCE-OP-EXPLOIT allows the proof to assume that the operation succeeds and produces a value v :

$$\frac{\rho[\vec{r}] = \vec{v} \quad f[pc] = \langle dst := \text{op } \vec{r} \rightarrow pc' \rangle \quad \forall v, \lceil \text{eval_op}(\text{op}, \vec{v}) = \text{Some}(v) \rceil \multimap c \lesssim_{i'} (\sigma, \text{State}(f, pc', \rho[dst \mapsto v])) \{\Phi\}}{c \lesssim_i (\sigma, \text{State}(f, pc, \rho)) \{\Phi\}} \text{ SOURCE-OP-EXPLOIT}$$

5.3 Exploiting Memory Operations

In separation logic, memory resources cannot be obtained without explicit allocation. However, when programs manipulate values related through a memory injection $\text{meminj}_E(I)$, heap assertions can be extracted directly from the global injection invariant.

To justify the soundness of the SOURCE-LOAD-EXPLOIT rule presented in fig. 12, suppose the source program is about to execute a load on an address v that is related to a target value w through $w \bowtie_I v$. The proof sketch proceeds in two steps:

1. **Pointer validity:** By case analysis, if v is not a valid pointer $\text{VPtr}(l)$, the source becomes stuck and the proof is immediate via FSOURCESTUCK. Otherwise, $v = \text{VPtr}(l)$, and $w \bowtie_I v$ ensures that $w = \text{VPtr}(l')$ with $(l', l) \in I$.
2. **Active status:** Opening the memory injection $\text{meminj}_\emptyset(I)$ via the extraction lemma 6 reveals that the pair (l', l) is either actively allocated or already freed. If the location were freed, attempting a load in the source would produce a stuck configuration. Thus, the freed case is instantly discharged, and the cell is guaranteed to be currently allocated.

From a user perspective, this rule exhibits an interesting asymmetry. In standard separation logic, and in our TARGET-LOAD rule, executing a load demands an existing points-to assertion in the context. In contrast, SOURCE-LOAD-EXPLOIT does not consume any points-to assertion. Instead, it extracts and provides both points-to assertions $l' \rightarrow_t a * l \rightarrow_s b$ from the global invariant $\text{meminj}_\emptyset(I)$, adding (l', l) to the exception set. For clarity, fig. 12 displays a simplified rule where the exception set is initially empty. Similar exploitation principles apply to the remaining memory operations.

$$\frac{\begin{array}{c} f[pc] = \langle dst := !src \rightarrow pc' \rangle \\ \rho[src] = v \quad w \bowtie_I v \quad \text{meminj}_\emptyset(I) \\ \left(\begin{array}{c} \forall l' l a b, v = \text{VPtr}(l) \wedge w = \text{VPtr}(l') \implies \\ l' \rightarrow_t a * l \rightarrow_s b * \lceil a \bowtie_I b \rceil * \text{meminj}_{\{(l', l)\}}(I) \multimap \\ c \lesssim_{i'} (\sigma, \text{State}(f, pc', \rho[dst \mapsto b])) \{\Phi\} \end{array} \right) \end{array}}{c \lesssim_i (\sigma, \text{State}(f, pc, \rho)) \{\Phi\}} \text{ SOURCE-LOAD-EXPLOIT}$$

Figure 12: Memory exploitation rule for source load instructions.

5.4 Synchronous Rules

While asynchronous and exploit rules provide flexibility for transformations that reorder or eliminate instructions, many compiler passes preserve the overall control flow of the program. For such passes, stepping target and source programs independently introduces redundant bookkeeping. By composing source exploit rules with target stepping rules, we obtain synchronous rules that execute matching instructions in lockstep.

This is notably the case when reasoning about synchronous function calls. To ensure modularity, we avoid explicitly stepping through the execution of the callees. Instead, we rely on the Hoare quadruples introduced in section 4.3. The synchronous call rule BOTH-CALL enables replacing a pair of matching function calls by their relational specification. Its structure highlights the core benefits of our relational separation logic.

Suppose both programs are about to execute a call:

- the target calls function f'_t with argument values $\vec{v}_t$, where $f_t[pc_t] = \langle dst_t := \text{call } f'_t(\vec{a}_t) \rightarrow pc'_t \rangle$ and $\rho_t[\vec{a}_t] = \vec{v}_t$;
- the source calls function f'_s with argument values $\vec{v}_s$, where $f_s[pc_s] = \langle dst_s := \text{call } f'_s(\vec{a}_s) \rightarrow pc'_s \rangle$ and $\rho_s[\vec{a}_s] = \vec{v}_s$;
- a relational specification $\{P\} f'_t \lesssim f'_s \{Q\}$ has already been established for the called functions.

Under these conditions, the BOTH-CALL rule proceeds as follows:

1. The caller must prove that current heap resources satisfy the relational precondition $P(\vec{v}_t, \vec{v}_s)$.

2. Instead of stepping through the internal instructions of f'_t and f'_s , the rule bypasses the calls entirely.
3. Execution resumes at the return points pc'_t and pc'_s with the destination registers updated with return values, satisfying the postcondition $Q(v_t, v_s)$.

Because Hoare quadruples are enclosed in the persistence modality, they represent duplicable specifications that can be verified once per function, and reused across many call sites. This provides horizontal modularity: functions in a program can be compiled and verified independently, and their correctness guarantees compose cleanly through BOTH-CALL.

6 Case Studies

To evaluate the relational separation logic developed in this work, we present three case studies: a local transformation exploiting undefined behavior, a loop transformation hoisting an invariant load, and a control-flow optimization eliminating redundant jump chains.

6.1 Undefined Behavior Exploitation

We begin with a local program transformation that eliminates a redundant zero-check after an arithmetic division. Consider the source and target code fragments in fig. 13.

Source Program		Target Program
1 : $r_1 := \text{Div } [r_n, r_d] \rightarrow 2$	$\implies$	1 : $r_1 := \text{Div } [r_n, r_d] \rightarrow 2$
2 : $r_b := \text{EqZ } [r_d] \rightarrow 3$		2 : $r_b := \text{ImmBool}(\text{false}) \rightarrow 3$
3 : ...		3 : ...

Figure 13: Exploiting division undefined behavior to simplify a zero test.

In the source program, instruction 1 divides the content of r_n by r_d , storing the quotient in r_1 . Instruction 2 tests whether the divisor r_d is zero via EqZ , storing the resulting boolean in r_b . In the target program, instruction 2 replaces this test with an immediate assignment of false.

The soundness of this transformation relies on the asymmetric nature of our simulation relation. When reasoning about instruction 1, we apply the rule SOURCE-OP-EXPLOIT. Because stuck source executions satisfy the simulation relation by FSOURCESTUCK, the rule provides the hypothesis that the operation succeeds: $\text{eval_op}(\text{Div}, [\rho_s(r_n), \rho_s(r_d)]) = \text{Some}(v)$. Under our operational semantics, division succeeds only when the divisor $\rho_s(r_d) = \text{VInt}(z_d)$ satisfies $z_d \neq 0$. Assuming that target and source register banks agree initially, the target division executes identically, advancing both programs to $pc = 2$.

At program point 2, the source program tests register r_d for equality to zero. Because $\rho_s[r_d] = \text{VInt}(z_d)$ with $z_d \neq 0$, this evaluation deterministically yields false. In the target program, the immediate load executes via TARGET-OP and writes false into r_b without inspecting r_d . Both programs reach $pc = 3$ in matching register and memory states, establishing the soundness of this transformation.

6.2 Load Hoisting

We next examine a transformation hoisting a load out of a loop. Figure 14 displays two versions of a factorial function $\text{fact}(n, \text{addr})$. The function initializes the memory cell at addr to 1, and computes the factorial of n .

In the unoptimized function fact_slow , the loop body performs a redundant heap load at $pc = 5$ on each iteration, reading the constant 1 stored at the address in r_{addr} to decrement r_n . The optimized function fact_fast hoists the load into the prelude. It further replaces the memory access with an immediate assignment of 1 into register r_{one} .

To prove the soundness of this transformation, we prove the following relational Hoare quadruple for all heap bijections I :

$$\{\text{same_args}_I\} \text{fact_fast} \lesssim \text{fact_slow} \{\text{same_res}_I\} \quad (3)$$

where same_args_I and same_res_I relate input arguments and return values under an extendable heap bijection:

$$\begin{aligned} \text{same_args}_I(\vec{v}_t, \vec{v}_s) &\triangleq \ulcorner \text{Forall2}(\bowtie_I, \vec{v}_t, \vec{v}_s) \urcorner * \text{meminj}_{\emptyset}(I) \\ \text{same_res}_I(v_t, v_s) &\triangleq \exists I', \ulcorner I \subseteq I' \urcorner * \ulcorner v_t \bowtie_{I'} v_s \urcorner * \text{meminj}_{\emptyset}(I') \end{aligned}$$

Source Function (fact_slow)	Target Function (fact_fast)
0 : $r_{res} := \text{ImmInt}(1) \rightarrow 1$	0 : $r_{res} := \text{ImmInt}(1) \rightarrow 1$
1 : $!r_{addr} := r_{res} \rightarrow 2$	1 : $!r_{addr} := r_{res} \rightarrow 2$
2 : $r_{cond} := \text{EqZ}[r_n] \rightarrow 3$	2 : $r_{one} := \text{ImmInt}(1) \rightarrow 3$
3 : if r_{cond} then 7 else 4	3 : $r_{cond} := \text{EqZ}[r_n] \rightarrow 4$
4 : $r_{res} := \text{Mul}[r_{res}, r_n] \rightarrow 5$	4 : if r_{cond} then 7 else 5
5 : $r_{one} := !r_{addr} \rightarrow 6$	5 : $r_{res} := \text{Mul}[r_{res}, r_n] \rightarrow 6$
6 : $r_n := \text{Sub}[r_n, r_{one}] \rightarrow 2$	6 : $r_n := \text{Sub}[r_n, r_{one}] \rightarrow 3$
7 : ret r_{res}	7 : ret r_{res}

Figure 14: Source and target factorial functions illustrating load hoisting.

The precondition $\text{same_args}_I(\vec{v}_t, \vec{v}_s)$ asserts that input arguments are pairwise related under I , alongside a complete ($E = \emptyset$) memory injection $\text{meminj}_\emptyset(I)$. The postcondition $\text{same_res}_I(v_t, v_s)$ guarantees that return values are related under an extended bijection $I' \supseteq I$, with the memory injection $\text{meminj}_\emptyset(I')$ fully restored. Allowing the bijection to grow monotonically accounts for any fresh memory allocations performed during execution.

The proof of eq. (3) begins by stepping through the prelude. Both programs initialize r_{res} to 1. Applying SOURCE-STORE-EXPLOIT extracts (l_t, l_s) from $\text{meminj}_\emptyset(I)$, yielding $l_s \rightarrow_s \text{VInt}(1)$ and advancing the source to $pc_s = 2$. Applying TARGET-STORE then executes the target store, producing $l_t \rightarrow_t \text{VInt}(1)$. Finally, the target initializes r_{one} via TARGET-OP and reaches $pc_t = 3$, establishing the following loop invariant:

- The target program is executing `fact_fast` at program counter $pc_t = 3$ with register bank ρ_t .
- The source program is executing `fact_slow` at program counter $pc_s = 2$ with register bank ρ_s .
- The register banks agree on the counter and accumulator: $\rho_t(r_n) \bowtie_I \rho_s(r_n)$ and $\rho_t(r_{res}) \bowtie_I \rho_s(r_{res})$.
- The registers r_{addr} hold related pointer values: $\rho_t(r_{addr}) = \text{VPtr}(l_t)$ and $\rho_s(r_{addr}) = \text{VPtr}(l_s)$ with $(l_t, l_s) \in I$.
- The target register r_{one} holds the constant $\text{VInt}(1)$ in ρ_t .
- The memory state satisfies $l_t \rightarrow_t \text{VInt}(1) * l_s \rightarrow_s \text{VInt}(1) * \text{meminj}_{\{(l_t, l_s)\}}(I)$, isolating the hoisted cell (l_t, l_s) while the remaining heap is governed by the injection.

Because the programs contain a loop and may diverge (when $n < 0$), the proof proceeds by coinduction. Our logic framework establishes semantic equivalence regardless of termination. Evaluating the zero-test $\text{EqZ}(r_n) = b$ synchronously in both programs yields two cases:

Base case ($b = \text{true}$): Both programs branch to the return instruction at $pc = 7$. The points-to assertions $l_t \rightarrow_t \text{VInt}(1) * l_s \rightarrow_s \text{VInt}(1)$ are returned to the memory injection via the reintegration lemma, restoring $\text{meminj}_\emptyset(I)$. Both programs execute `ret  $r_{res}$` , satisfying the postcondition same_res_I with $I' = I$.

Step case ($b = \text{false}$): Both programs execute the multiplication $r_{res} := r_{res} \times r_n$. At source $pc_s = 5$, the source load $r_{one} := !r_{addr}$ executes via SOURCE-LOAD directly from $l_s \rightarrow_s \text{VInt}(1)$ without requiring memory exploitation. Both programs then decrement $r_n := r_n - r_{one}$, preserving register agreement, and branch back to the loop headers ($pc_t = 3$ and $pc_s = 2$). This re-establishes the invariant, closing the case by the coinductive hypothesis.

6.3 Nop Tunneling

We now consider an optimization on arbitrary control-flow graphs present in CompCert [Ler09a]: nop tunneling. Intermediate compiler passes frequently introduce redundant jump instructions (`nop  $\rightarrow$   $pc$`). A sequence of consecutive jumps forms a *nop chain*:

$$pc_0 \rightarrow pc_1 \rightarrow pc_2 \rightarrow \dots \rightarrow pc_k \rightarrow pc_{\text{final}}$$

where each node pc_m ($0 \leq m < k$) contains `nop  $\rightarrow$   $pc_{m+1}$` , and pc_{final} is the first non-nop instruction. Nop tunneling short-circuits these chains by redirecting branch targets directly to pc_{final} , reducing instruction count and eliminating superfluous jump overhead.

The transformation is defined in Rocq over an arbitrary μ RTL function f . To ensure termination, even in the presence of cyclic jump chains (such as an infinite loop $pc: \text{nop} \rightarrow pc$), the computation of the final successor of an instruction at pc is parameterized by a recursion depth $d \in \mathbb{N}$:

$$\begin{aligned} \text{find_next_pc}(0, f, pc) &\triangleq pc \\ \text{find_next_pc}(1 + d, f, pc) &\triangleq \begin{cases} \text{find_next_pc}(d, f, pc') & \text{if } f[pc] = \langle \text{nop} \rightarrow pc' \rangle \\ pc & \text{otherwise} \end{cases} \end{aligned}$$

The optimization $\text{tunnel_nop}(d, f)$ maps find_next_pc over every instruction in the control-flow graph of f , redirecting each successor label pc to $\text{find_next_pc}(d, f, pc)$. The function entry point is updated in the same manner. To establish the soundness of nop tunneling for an arbitrary function f , we prove the following relational Hoare quadruple for all depths d and heap bijections I :

$$\{\text{same_args}_I\} \text{tunnel_nop}(d, f) \lesssim f \{\text{same_res}_I\} \quad (4)$$

The proof relies on a coinductive simulation invariant. Given a fixed initial fuel $depth$, the invariant asserts that at any step of execution:

- The target program is executing the optimized function $\text{tunnel_nop}(depth, f)$ at program counter pc_t with register bank ρ_t .
- The source program is executing the original function f at program counter pc_s with register bank ρ_s .
- The target program counter is the tunneled successor of the source program counter with remaining fuel d : $pc_t = \text{find_next_pc}(d, f, pc_s)$.
- The register banks agree on all registers: $\forall r, \rho_t(r) \bowtie_I \rho_s(r)$.
- The memory heaps are fully synchronized under a monotone bijection I : $\text{meminj}_\emptyset(I)$.

The proof of eq. (4) proceeds by coinduction followed by an inner induction on the remaining fuel d :

Base case ($d = 0$): When fuel is exhausted, $\text{find_next_pc}(0, f, pc_s) = pc_s$, so $pc_t = pc_s$. The target and source are positioned at identical program points. By case analysis on the instruction at pc_s , both programs progress in lockstep for each instruction kind. The next target program counter is the redirected label $\text{find_next_pc}(depth, f, pc'_s)$, which re-establishes the invariant with full fuel $d = depth$. The coinductive hypothesis closes this case.

Inductive step ($d > 0$): We inspect the source instruction at pc_s :

- If pc_s maps to a jump $\text{nop} \rightarrow pc'_s$, then $\text{find_next_pc}(1 + d', f, pc_s) = \text{find_next_pc}(d', f, pc'_s)$. The source executes the jump via SOURCE-NOP, advancing to pc'_s . The target remains at pc_t . The inner induction hypothesis on d' concludes this case.
- If pc_s does not refer to a jump instruction, $\text{find_next_pc}(1 + d', f, pc_s) = pc_s$, so $pc_t = pc_s$. As in the base case, target and source progress in lockstep, resetting fuel to $depth$ for successors. The coinductive hypothesis closes this case.

When verifying function call instructions, the proof applies the synchronous BOTH-CALL rule. Because the coinductive invariant is parameterized over arbitrary μ RTL functions, it can be instantiated directly for the callee. Therefore, the relational Hoare quadruple required by BOTH-CALL is discharged without additional lemmas. This illustrates the modularity of our relational separation logic: function-level optimizations are verified in isolation and compose cleanly across whole-program call graphs.

7 Related Work

We compare our approach with existing verified compilers, simulation techniques, and relational separation logics.

7.1 Verified Compilers

Non-Determinism CompCert [Ler09b] is a verified C compiler that establishes correctness by proving forward simulation diagrams for each compilation phase. When the target language is deterministic, a forward simulation implies a backward simulation, which guarantees semantic preservation. However, extensions such as shared memory concurrency introduce internal non-determinism into target executions, requiring backward simulations to be established directly. CakeML [KMNO14; Tan+19] avoids this complication by enforcing determinism throughout its source semantics and

intermediate representations, disallowing concurrency and internal non-deterministic choice. In our framework, we directly establish a safe backward simulation. This enables our system to accommodate internal non-determinism in the target language. Nevertheless, our current μ RTL operational semantics features only limited non-determinism, specifically fresh memory allocation, with no support for concurrent primitives.

Stuttering CompCert handles stuttering by indexing simulation relations with an explicit well-founded order, which must strictly decrease on stuttering steps. CakeML uses a fuel-indexed operational semantics. Because evaluation in CakeML is deterministic, semantic preservation is established by case analysis on the existence of a sufficient fuel value to reach termination. In our approach, we address stuttering coinductively through a free simulation relation [Cho+23]. This formulation avoids constructing explicit well-founded metrics or managing fuel bounds.

Memory Models CompCert uses a block-based memory model [LB08], in which program transformations that alter memory layouts are justified through global memory injections. These injections map source memory blocks to target blocks across the global heap. CakeML similarly tracks memory through a global mutable store, lowering this store to a flat linear array in the backend. In our work, we employ separation logic to reason locally about memory fragments and memory injections. Local heap assertions avoid reasoning globally about the entire memory state. We anticipate that this formulation will facilitate future proofs of transformations that modify memory.

Modularity CompCert is structured as a whole-program compiler with global call-stack invariants. To support verified separate compilation and modular linking, several extensions have been developed [SBCA15; Kan+16]. Similarly, while early CakeML proofs were whole-program, recent extensions support dynamic compilation primitives [Sew+23]. In our framework, relational Hoare quadruples provide horizontal compositionality, allowing verified code components to be reasoned about independently. This property is particularly useful when stating the invariant of the nop-tunneling pass, in contrast to the whole-program simulation invariants used in CompCert.

7.2 Relational Separation Logics

Logical Foundations Simuliris [Gäh+22] is a relational separation logic framework implemented within Iris [Jun+18]. Recent work on Tail Modulo Cons (TMC) by Allain et al. [All+25] similarly embeds a relational separation logic in Iris to verify the soundness of the TMC transformation for a functional language. Other relational frameworks built on Iris include ReLoC [FKB21] for contextual refinement in concurrent languages, and Melocoton [Gué+23] for verified interoperability between OCaml and C.

While Iris provides substantial expressive power, its semantic model relies on complex constructions, such as step-indexed resource algebras and modal update operators. In our framework, we do not build upon the semantic model of Iris. Instead, we formulate our relational separation logic directly over concrete heap states and coinductive simulation relations. Our logic definitions remain direct while maintaining compatibility with MoSeL [Kre+18], the interactive separation logic proof mode.

Program Representation and Control Flow The nature of the underlying languages also distinguishes our work from prior relational separation logics. Both Simuliris and TMC target high-level languages with structured, syntax-directed control flow. In contrast, our framework supports μ RTL, a low-level register transfer language where programs are represented as control-flow graphs. Because arbitrary control-flow graphs can contain unstructured loops, reasoning about execution steps relies heavily on coinduction, necessitating proof rules that interact cleanly with coinductive reasoning.

Verified Transformations The scope of verified program transformations also differs. Simuliris focuses on fine-grained concurrent optimizations, such as code motion and instruction reordering under weak memory models. TMC proves the correctness of a specialized source-to-source transformation, converting non-tail-recursive functions into tail-recursive loops by mutating newly allocated constructor fields. In our work, we evaluate our logic on backend compiler transformations over control-flow graphs, such as the optimization that eliminates redundant jump chains.

8 Contributions & Future Work

Due to space constraints, refer to page 2 for a summary of contributions and a discussion of further work.

References

- [All+25] Clément Allain, Frédéric Bour, Basile Clément, François Pottier, and Gabriel Scherer. “Tail Modulo Cons, OCaml, and Relational Separation Logic.” In: *Proc. ACM Program. Lang.* 9.POPL (2025), pp. 2337–2363.
- [Alm+17] José Bacelar Almeida et al. “Jasmin: High-Assurance and High-Speed Cryptography.” In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*. Ed. by Bhavani Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu. ACM, 2017, pp. 1807–1823.
- [Cha20] Arthur Charguéraud. “Separation logic for sequential programs (functional pearl).” In: *Proc. ACM Program. Lang.* 4.ICFP (2020), 116:1–116:34.
- [Cho+23] Minki Cho, Youngju Song, Dongjae Lee, Lennard Gäher, and Derek Dreyer. “Stuttering for Free.” In: *Proc. ACM Program. Lang.* 7.OOPSLA2 (2023), pp. 1677–1704.
- [FKB21] Dan Frumin, Robbert Krebbers, and Lars Birkedal. “ReLoC Reloaded: A Mechanized Relational Logic for Fine-Grained Concurrency and Logical Atomicity.” In: *Log. Methods Comput. Sci.* 17.3 (2021).
- [Gäh+22] Lennard Gäher, Michael Sammler, Simon Spies, Ralf Jung, Hoang-Hai Dang, Robbert Krebbers, Jeehoon Kang, and Derek Dreyer. “Simuliris: a separation logic framework for verifying concurrent program optimizations.” In: *Proc. ACM Program. Lang.* 6.POPL (2022), pp. 1–31.
- [Gim98] Eduardo Giménez. *A Tutorial on Recursive Types in Coq*. Research Report RT-0221. Projet COQ. INRIA, May 1998, p. 42.
- [Gué+23] Armaël Guéneau, Johannes Hostert, Simon Spies, Michael Sammler, Lars Birkedal, and Derek Dreyer. “Melo-coton: A Program Logic for Verified Interoperability Between OCaml and C.” In: *Proc. ACM Program. Lang.* 7.OOPSLA2 (2023), pp. 716–744.
- [Jun+18] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. “Iris from the ground up: A modular foundation for higher-order concurrent separation logic.” In: *J. Funct. Program.* 28 (2018), e20.
- [Jun+26] Ralf Jung et al. *std++: An extended standard library for Rocq*. 2026.
- [Kan+16] Jeehoon Kang, Yoonseung Kim, Chung-Kil Hur, Derek Dreyer, and Viktor Vafeiadis. “Lightweight verification of separate compilation.” In: *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*. Ed. by Rastislav Bodík and Rupak Majumdar. ACM, 2016, pp. 178–190.
- [KMNO14] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. “CakeML: a verified implementation of ML.” In: *The 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL ’14, San Diego, CA, USA, January 20-21, 2014*. Ed. by Suresh Jagannathan and Peter Sewell. ACM, 2014, pp. 179–192.
- [KN06] Gerwin Klein and Tobias Nipkow. “A machine-checked model for a Java-like language, virtual machine, and compiler.” In: *ACM Trans. Program. Lang. Syst.* 28.4 (2006), pp. 619–695.
- [Kre+17] Robbert Krebbers, Ralf Jung, Ales Bizjak, Jacques-Henri Jourdan, Derek Dreyer, and Lars Birkedal. “The Essence of Higher-Order Concurrent Separation Logic.” In: *Programming Languages and Systems - 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-29, 2017, Proceedings*. Ed. by Hongseok Yang. Vol. 10226. Lecture Notes in Computer Science. Springer, 2017, pp. 696–723.
- [Kre+18] Robbert Krebbers, Jacques-Henri Jourdan, Ralf Jung, Joseph Tassarotti, Jan-Oliver Kaiser, Amin Timany, Arthur Charguéraud, and Derek Dreyer. “MoSeL: a general, extensible modal framework for interactive proofs in separation logic.” In: *Proc. ACM Program. Lang.* 2.ICFP (2018), 77:1–77:30.
- [LB08] Xavier Leroy and Sandrine Blazy. “Formal Verification of a C-like Memory Model and Its Uses for Verifying Program Transformations.” In: *J. Autom. Reason.* 41.1 (2008), pp. 1–31.
- [Ler09a] Xavier Leroy. “A Formally Verified Compiler Back-end.” In: *J. Autom. Reason.* 43.4 (2009), pp. 363–446.
- [Ler09b] Xavier Leroy. “Formal verification of a realistic compiler.” In: *Commun. ACM* 52.7 (2009), pp. 107–115.
- [Mil89] Robin Milner. *Communication and concurrency*. PHI Series in computer science. Prentice Hall, 1989. ISBN: 978-0-13-115007-2.
- [Pou16] Damien Pous. “Coinduction All the Way Up.” In: *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science, LICS ’16, New York, NY, USA, July 5-8, 2016*. Ed. by Martin Grohe, Eric Koskinen, and Natarajan Shankar. ACM, 2016, pp. 307–316.
- [Pou26] Damien Pous. *Coinduction: a library for doing proofs by (enhanced) coinduction*. Version 1.22. 2026.

- [Rey02] John C. Reynolds. “Separation Logic: A Logic for Shared Mutable Data Structures.” In: *17th IEEE Symposium on Logic in Computer Science (LICS 2002)*, 22-25 July 2002, Copenhagen, Denmark, *Proceedings*. IEEE Computer Society, 2002, pp. 55–74.
- [San12] Davide Sangiorgi. *Introduction to Bisimulation and Coinduction*. Cambridge University Press, 2012. ISBN: 978-1-107-00363-7.
- [SBCA15] Gordon Stewart, Lennart Beringer, Santiago Cuéllar, and Andrew W. Appel. “Compositional CompCert.” In: *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*. Ed. by Sriram K. Rajamani and David Walker. ACM, 2015, pp. 275–287.
- [Sew+23] Thomas Sewell, Magnus O. Myreen, Yong Kiam Tan, Ramana Kumar, Alexander Mihajlovic, Oskar Abrahams-son, and Scott Owens. “Cakes That Bake Cakes: Dynamic Computation in CakeML.” In: *Proc. ACM Program. Lang.* 7.PLDI (2023), pp. 1121–1144.
- [SS17] Steven Schäfer and Gert Smolka. “Tower Induction and Up-to Techniques for CCS with Fixed Points.” In: *Relational and Algebraic Methods in Computer Science - 16th International Conference, RAMiCS 2017, Lyon, France, May 15-18, 2017, Proceedings*. Ed. by Peter Höfner, Damien Pous, and Georg Struth. Vol. 10226. Lecture Notes in Computer Science. 2017, pp. 274–289.
- [Tan+19] Yong Kiam Tan, Magnus O. Myreen, Ramana Kumar, Anthony C. J. Fox, Scott Owens, and Michael Norrish. “The verified CakeML compiler backend.” In: *J. Funct. Program.* 29 (2019), e2.
- [The26] The Rocq Development Team. *The Rocq Prover*. Formerly known as The Coq Proof Assistant. Inria. 2026.
- [Xia+20] Li-yao Xia, Yannick Zakowski, Paul He, Chung-Kil Hur, Gregory Malecha, Benjamin C. Pierce, and Steve Zdancewic. “Interaction trees: representing recursive and impure programs in Coq.” In: *Proc. ACM Program. Lang.* 4.POPL (2020), 51:1–51:32.
- [Yan07] Hongseok Yang. “Relational separation logic.” In: *Theor. Comput. Sci.* 375.1-3 (2007), pp. 308–334.

Contents

1	Verified Compilation Background	3
1.1	Small-Step Operational Semantics	3
1.2	Program Behaviors	3
1.3	Program Refinement	3
1.4	Simulation Relations	4
1.5	Coinductive Predicates	4
1.6	Simulation Schemes & Stuttering	5
2	The Free Simulation	7
2.1	Implicit Simulation	7
2.2	Step-Indexed Free Simulation	7
2.3	Properties	9
3	The Register Transfer Language (μRTL)	9
3.1	Values, Registers, and Memory	10
3.2	Syntax and Program Structure	10
3.3	Small-Step Operational Semantics	10
3.4	Stack Modularity and Execution Invariants	12
4	Relational Separation Logic	12
4.1	Separation Logic	12
4.2	Relational Assertions & Memory Predicates	13
4.3	Relational Hoare Quadruples	14
5	μRTL Proof Rules	15
5.1	Asynchronous Rules	15
5.2	Undefined Behavior Exploitation	15
5.3	Exploiting Memory Operations	16
5.4	Synchronous Rules	16
6	Case Studies	17
6.1	Undefined Behavior Exploitation	17
6.2	Load Hoisting	17
6.3	Nop Tunneling	18
7	Related Work	19
7.1	Verified Compilers	19
7.2	Relational Separation Logics	20
8	Contributions & Future Work	20
	References	21
A	Coinduction Example: Stream Bisimilarity	24
B	Free Simulation Example	24

A Coinduction Example: Stream Bisimilarity

To illustrate how coinductive reasoning operates in practice, consider the *bisimilarity* relation $\approx$ on streams of natural numbers, $\text{Stream}(\mathbb{N})$, defined coinductively by the following inference rule:

$$\frac{n_1 = n_2 \quad s_1 \approx s_2}{\text{scons}(n_1, s_1) \approx \text{scons}(n_2, s_2)} \text{ Bisim-Step}$$

Let us prove that this relation is reflexive:

$$\forall s \in \text{Stream}(\mathbb{N}), s \approx s.$$

The proof proceeds by coinduction on the *bisimilarity* relation:

1. Applying the coinduction principle provides the coinductive hypothesis:

$$\text{ColH} : \forall s' \in \text{Stream}(\mathbb{N}), s' \approx s',$$

with the restriction that ColH may only be applied under a constructor of the coinductive relation.

2. We decompose the stream s into a head $n \in \mathbb{N}$ and a tail $s' \in \text{Stream}(\mathbb{N})$, so that $s = \text{scons}(n, s')$. The goal becomes:

$$\text{scons}(n, s') \approx \text{scons}(n, s').$$

3. We apply the coinductive rule Bisim-Step. The first premise $n = n$ holds trivially, leaving the second premise $s' \approx s'$ to be established.
4. Because the constructor Bisim-Step has been applied, we can now legitimately invoke ColH. This immediately concludes the proof.

B Free Simulation Example

To illustrate how the free simulation operates in practice, we consider a small imperative language with commands $\text{skip} \mid c_1; c_2 \mid \text{while } b \{c\} \mid \text{ret } v$, where $\text{ret } v$ is a terminal configuration. The small-step reduction relation $c \rightarrow c'$ is defined by:

$$\begin{array}{ll} \text{skip}; c \rightarrow c & \\ \text{while } b \{c\}; k \rightarrow c; \text{while } b \{c\}; k & (\text{if } b = \text{true}) \\ \text{while } b \{c\}; k \rightarrow k & (\text{if } b = \text{false}) \end{array}$$

We instantiate the index domains with the natural numbers, $I = J = (\mathbb{N}, <)$, and define the target and source programs:

$$\begin{aligned} P_t &\triangleq \text{while } (b) \{ \text{skip} \}; \text{ret } () \\ P_s &\triangleq \text{while } (b) \{ \text{skip}; \text{skip}; \text{skip} \}; \text{ret } () \end{aligned}$$

We wish to prove that $P_t \approx_{\leq 0} P_s \{=\}$. We proceed by coinduction on the free simulation. This gives us the following coinductive hypothesis:

$$\forall j > 0, \forall i > 0, \quad P_t \approx_{j \leq i} P_s \{=\}.$$

We then perform case analysis on the boolean condition b :

- If $b = \text{false}$, both programs exit their loops in a single step: $P_t \rightarrow \text{ret } ()$ and $P_s \rightarrow \text{ret } ()$. Applying FTARGETSTEPS and FSOURCESTEPS, both configurations reach $\text{ret } ()$, which are final values returning $()$. The goal is thus closed immediately by FRELATED.
- If $b = \text{true}$, using FTARGETSTEPS the target loop unrolls: $P_t \rightarrow \text{skip}; P_t$, and the target index is updated to $j = 1$. Again with FTARGETSTEPS, the target reduces: $\text{skip}; P_t \rightarrow P_t$, and the target index is increased to $j = 2$. The simulation goal becomes $P_t \approx_{2 \leq 0} P_s \{=\}$.

The source loop then executes four steps: first, $P_s \rightarrow \text{skip}; \text{skip}; \text{skip}; P_s$, updating the source index to $i = 1$ via FSOURCESTEPS; next, three successive applications of FSOURCESTEPS consume the three skip statements, yielding index $i = 4$ and state P_s . The simulation goal becomes $P_t \approx_{2 \leq 4} P_s \{=\}$.

Having accumulated positive progress indices ($0 < 2$ and $0 < 4$), we can apply the coinductive hypothesis to conclude the proof.

$$\begin{array}{c}
\frac{0 < 2 \quad 0 < 4}{P_t \lesssim_4 P_s \{=\}} \text{CIH} \\
\hline
\frac{P_t \lesssim_1 \text{skip; skip; skip; } P_s \{=\}}{P_t \lesssim_0 P_s \{=\}} \text{FSOURCESTEPS } (\times 3) \\
\hline
\frac{P_t \lesssim_0 P_s \{=\}}{\text{skip; } P_t \lesssim_0 P_s \{=\}} \text{FTARGETSTEPS} \\
\hline
P_t \lesssim_0 P_s \{=\}
\end{array}$$

Figure 15: Derivation tree for the loop iteration case ($b = \text{true}$).

The complete derivation tree for the loop iteration case ($b = \text{true}$) is displayed in fig. 15.

As illustrated by this example, the source and target programs can execute arbitrary numbers of asynchronous steps, at whatever pace is convenient, until matching invariants are reached.